

```
# =====
# TASK 6: KNN CLASSIFICATION USING IRIS DATASET
# =====

# =====
# STEP 1: IMPORT LIBRARIES
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

# =====
# STEP 2: LOAD DATASET
# =====

df = pd.read_csv("Iris.csv")

print("First 5 Rows")
print(df.head())

print("\nShape")
print(df.shape)

print("\nColumns")
print(df.columns)

# =====
# STEP 3: DATA PREPROCESSING
# =====

# Remove Id column if present

if "Id" in df.columns:
    df.drop("Id", axis=1, inplace=True)

# Encode Species column

le = LabelEncoder()

df["Species"] = le.fit_transform(df["Species"])

# =====
# STEP 4: FEATURES AND TARGET
# =====

X = df.drop("Species", axis=1)

y = df["Species"]

# =====
# STEP 5: TRAIN TEST SPLIT
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
```

```
# =====
# STEP 6: FEATURE SCALING
# =====

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

# =====
# STEP 7: TRAIN KNN MODEL
# =====

knn = KNeighborsClassifier(
    n_neighbors=5
)

knn.fit(X_train, y_train)

# =====
# STEP 8: PREDICTIONS
# =====

y_pred = knn.predict(X_test)

# =====
# STEP 9: ACCURACY
# =====

accuracy = accuracy_score(
    y_test,
    y_pred
)

print("\nAccuracy")
print(accuracy)

# =====
# STEP 10: CONFUSION MATRIX
# =====

cm = confusion_matrix(
    y_test,
    y_pred
)

print("\nConfusion Matrix")
print(cm)

# =====
# STEP 11: CLASSIFICATION REPORT
# =====

print("\nClassification Report")

print(
    classification_report(
        y_test,
        y_pred
    )
)

# =====
# STEP 12: HEATMAP
# =====

plt.figure(figsize=(6,4))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
```

```
    cmap='Blues'
)

plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()

# =====
# STEP 13: FIND BEST K
# =====

k_scores = []

for k in range(1, 21):

    model = KNeighborsClassifier(
        n_neighbors=k
    )

    model.fit(X_train, y_train)

    pred = model.predict(X_test)

    score = accuracy_score(
        y_test,
        pred
    )

    k_scores.append(score)

    print(
        "K =",
        k,
        "Accuracy =",
        round(score, 4)
    )

# =====
# STEP 14: K VS ACCURACY GRAPH
# =====

plt.figure(figsize=(8,5))

plt.plot(
    range(1,21),
    k_scores,
    marker='o'
)

plt.xlabel("K Value")
plt.ylabel("Accuracy")
plt.title("K Value vs Accuracy")

plt.grid(True)

plt.show()

# =====
# STEP 15: BEST K VALUE
# =====

best_k = k_scores.index(max(k_scores)) + 1

print("\nBest K =", best_k)
print("Best Accuracy =", max(k_scores))

# =====
# STEP 16: FINAL MESSAGE
# =====

print("\nTask 6 Completed Successfully")
```



```

First 5 Rows
  Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm  Species
0   1             5.1           3.5           1.4           0.2  Iris-setosa
1   2             4.9           3.0           1.4           0.2  Iris-setosa
2   3             4.7           3.2           1.3           0.2  Iris-setosa
3   4             4.6           3.1           1.5           0.2  Iris-setosa
4   5             5.0           3.6           1.4           0.2  Iris-setosa

Shape
(150, 6)

Columns
Index(['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm',
       'Species'],
      dtype='object')

```

```

Accuracy
1.0

```

```

Confusion Matrix

```

```

[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]

```

```

Classification Report

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11

accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

